CS405 Project 1 — Transformations, Projection & Curves from Paintings

Ezgi Aslantürk · 33937 · github.com/ezgiaslanturk/cs405-project1

1 · Painting

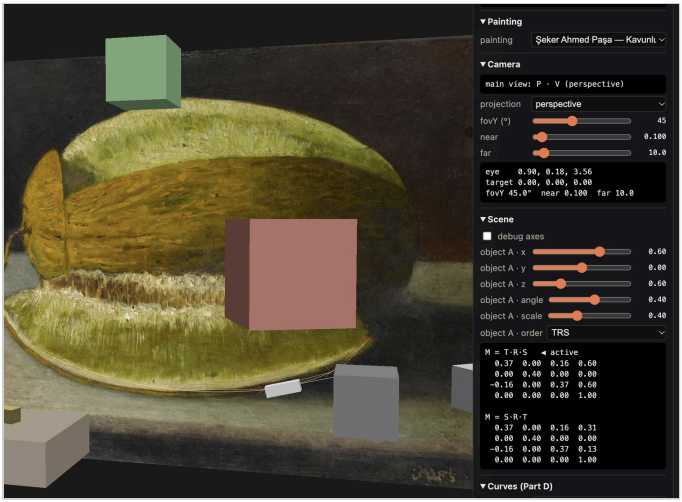
Şeker Ahmed Paşa — *Kavunlu Natürmort* (Still Life with Melons), Sakıp Sabancı Museum. I chose it first of all because I really like this painting. It also has strong organic contours: the long silhouette of the whole melon and the curved lower edge of the cut slice are smooth, closed shapes that a few cubic segments can follow. The scene is shallow and frontal, so the camera work happens in front of the canvas, not inside the painted space. The painting is a textured quad centred at the origin in the plane z = 0, with y ∈ [−1, 1] and x ∈ [−a, a], a = 1280 / 893 ≈ 1.433. Every object sits at z > 0.

2 · Part A — Transformations

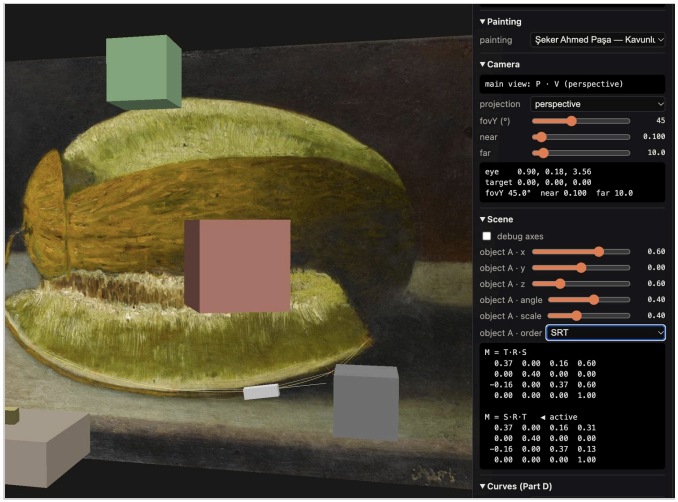
The whole application uses one convention: column-major `Float32Array(16)` matrices (element (r, c) at index 4c + r), column vectors p′ = M·p, and local transforms composed as **M = T·R·S**. `mat4.js` implements identity, multiply, translate, rotateX/Y/Z, scale, transpose and the rigid inverse. The rigid inverse uses the structure of M = T·R instead of a general 4×4 inverse:

$$M = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \quad \Rightarrow \quad M^{-1} = \begin{bmatrix} R^T & -R^T t \\ 0 & 1 \end{bmatrix}$$

Four cubes are placed in front of the painting (objects A, B, C and the object that follows the curve). Object A is driven from the UI: translation (x, y, z), rotation angle about y, uniform scale, and a switch between T·R·S and S·R·T. Both matrices are always printed in the panel, and the active one is marked.



S-A1 · M = T·R·S active: the red cube stays at t = (0.6, 0, 0.6).



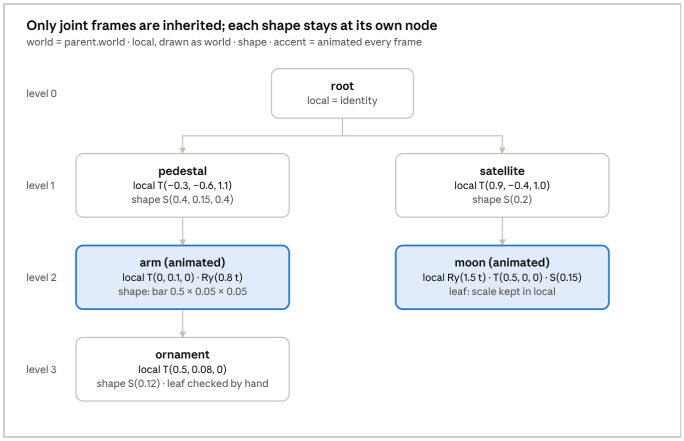
S-A2 · M = S·R·T active, same sliders: the cube moves to (0.31, 0, 0.13).

Why the order matters. Read right to left. T·R·S scales the cube about its own centre, turns it in place, then moves it to t. S·R·T moves it first, so the rotation and the scale then act on the offset too. With t = (0.6, 0, 0.6), θ = 0.4 rad and s = 0.4 (the values in S-A1/S-A2), the translation column of T·R·S is exactly t = (0.60, 0, 0.60). In S·R·T it becomes s·R(θ)·t = (0.31, 0, 0.13): the offset is turned by θ about the world y axis and shrunk to 40 %, so the cube jumps toward the origin. Changing the angle makes it orbit the origin, and changing the scale pulls it toward the origin. The upper-left 3×3 blocks of the two matrices are identical here, because a uniform scale commutes with a rotation. With a non-uniform scale they would also differ: R·S stretches along the cube's own axes, S·R along the world axes, which shears the rotated cube. The rest of the code uses T·R·S.

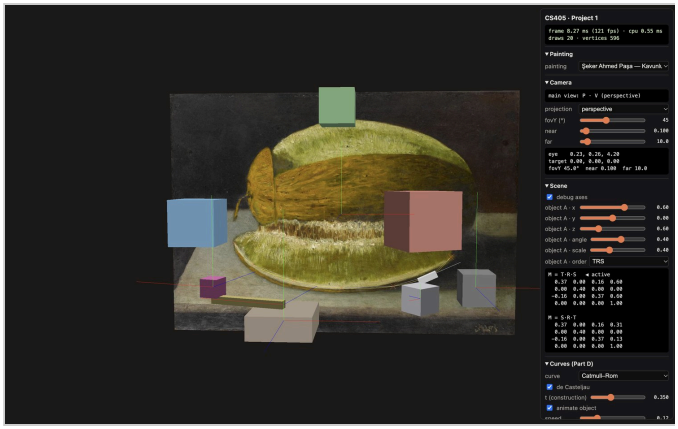
3 · Part B — Scene graph

`SceneNode` holds a local matrix, a parent, its children and a world matrix. `updateWorld(parentWorld)` sets world = parentWorld · local and recurses into the children; the root is called with no argument and uses the identity. The tree has six nodes and four levels (root → pedestal → arm → ornament). The arm turns about the pedestal's vertical axis every frame and carries the ornament with it; the moon orbits the satellite.

Joint frame in the chain, shape outside it. A node's `local` holds only T·R. The size of its cube is a separate `shape` matrix multiplied on the right when drawing (world · shape), so it is never inherited. A flat pedestal therefore does not squash the arm or the ornament. Only the moon, a leaf, keeps its scale in `local`. Debug mode draws every node's local axes at its world matrix.



Scene-graph tree. Turning the arm moves only the arm and the ornament; moving the pedestal moves its whole subtree.



S-B1 · debug axes on: each node's local axes (x red, y green, z blue) at its world matrix.

One leaf's world matrix, by hand. Take the ornament when the arm angle is θ = 90° (t = π / (2 · 0.8) ≈ 1.96 s). Its chain is

$$W_{\text{ornament}} = \underbrace{T(-0.3,\,-0.6,\,1.1)}_{\text{pedestal}}\;\underbrace{T(0,\,0.1,\,0)\,R_y(90^\circ)}_{\text{arm}}\;\underbrace{T(0.5,\,0.08,\,0)}_{\text{ornament}}$$

$R_y(90°)$ sends x̂ to (0, 0, −1) and ẑ to (1, 0, 0). The two translations on the left add to (−0.3, −0.5, 1.1). The ornament's own offset is rotated first: $R_y(90°)\cdot(0.5, 0.08, 0) = (0, 0.08, -0.5)$. So

$$W_{\text{ornament}} = \begin{bmatrix} 0 & 0 & 1 & -0.30 \\ 0 & 1 & 0 & -0.42 \\ -1 & 0 & 0 & 0.60 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The ornament's centre is at (−0.30, −0.42, 0.60): half a unit from the pivot, on the side facing the painting. The cube is then drawn with W · S(0.12).

4 · Part C — Camera and projection

The view matrix. The GPU has no camera, so the view matrix is the inverse of the camera's placement in the world. In view space the camera sits at the origin and looks down −z. Two cross products build an orthonormal basis from eye **e**, target **c** and the up hint **up**:

$$w = \frac{e - c}{\|e - c\|}, \qquad u = \frac{up \times w}{\|up \times w\|}, \qquad v = w \times u$$

w points backwards, because the camera looks along −w. u, v, w are the camera's axes in world coordinates, so the placement is $M_{cam} = T(e)\cdot R$ with R = [u v w]. R is orthonormal, so the rigid inverse of Part A gives the view matrix directly: the rows of V are u, v and w, and the translation is $-R^T e$.

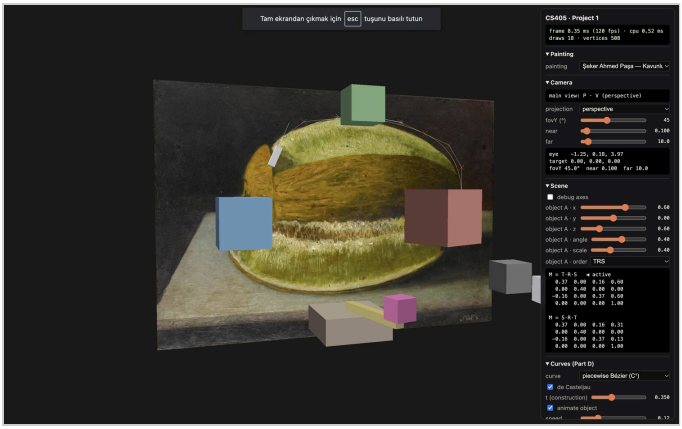
$$V = M_{cam}^{-1} = \begin{bmatrix} u_x & u_y & u_z & -u\cdot e \\ v_x & v_y & v_z & -v\cdot e \\ w_x & w_y & w_z & -w\cdot e \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Degenerate up. When up is parallel to the view direction, up × w = 0 and normalising it gives NaN, so the scene disappears. `lookAt` checks $|up \times w| < 10^{-6}$ and then uses (0, 1, 0), or (1, 0, 0) when w is itself almost vertical. The orbit control also clamps the pitch to ±(90° − 3°).

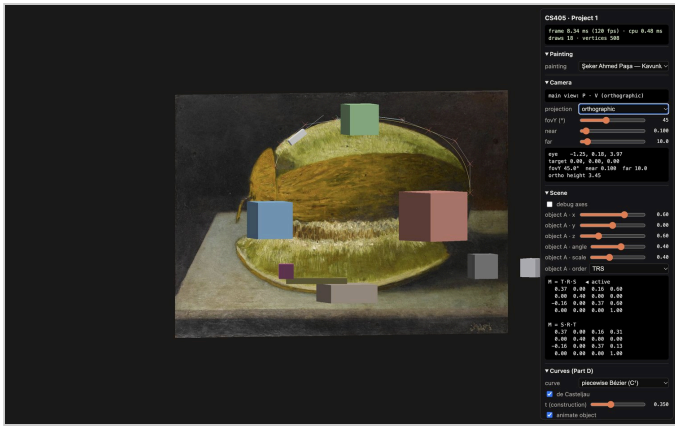
Perspective for WebGPU. With $f_y = 1 / \tan(\text{fovY} / 2)$, x and y are scaled by f_y / aspect and f_y, and row 4 copies −z into w for the divide. The third row has to map z = −n to 0 and z = −f to 1 after dividing by −z. Writing $z_{ndc} = (A\,z + B) / (-z)$:

$$\frac{-An + B}{n} = 0 \Rightarrow B = An, \qquad \frac{-Af + B}{f} = 1 \Rightarrow A = -\frac{f}{f-n}, \quad B = -\frac{fn}{f-n}$$

This is WebGPU's [0, 1] depth range. OpenGL's matrix would map the same interval to [−1, 1], and half the depth buffer would be clipped. The orthographic matrix maps the box [l, r] × [b, t] × [−n, −f] to the same NDC cube, with $z_{ndc} = -(z + n) / (f - n)$. It uses the same near and far. Its height follows the perspective view at the target ($2 \cdot \text{radius} \cdot \tan(\text{fovY} / 2)$), so switching keeps the scene the same size.



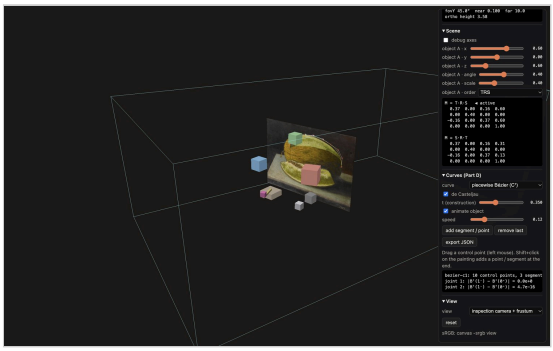
S-C1 · perspective, eye (−1.25, 0.18, 3.97).



S-C2 · orthographic, same camera (ortho height 3.45).

From the same eye on the left of the painting, the perspective view shows the canvas as a trapezoid: its near left edge is clearly taller than its far right edge, and the cubes closest to the camera look largest. In the orthographic view both edges of the canvas have the same height, parallel edges stay parallel, and an object's size no longer depends on its distance.

Inspection camera. A second, fixed camera at (5.5, 3.5, 7.5) looks at (0, 0, 1.2). In that view the first camera's frustum is drawn as lines between its eight corners: corner = e − w·d ± u·h·aspect ± v·h, with d ∈ {n, f} and h = d·tan(fovY / 2) (or the constant ortho half-height).



S-C3 · inspection view: the main camera's view volume in cyan (here orthographic, so a box).

Near and far. I use n = 0.1 and f = 10: the scene and the furthest orbit (radius 8) fit inside 10 units. Stored depth is hyperbolic in the view distance d:

$$z_{ndc}(d) = \frac{f\,(d-n)}{d\,(f-n)}$$

With these values, $z_{ndc}(1) = 0.909$, so the first unit in front of the camera uses 91 % of the depth range. The painting at the default distance of 2.8 already sits at 0.974. The smallest depth step that a 24-bit buffer separates is roughly $\Delta d \approx 2^{-24} \cdot d^2 / n$, which is $4.7 \cdot 10^{-6}$ units at d = 2.8. The curves are drawn 0.01 in front of the canvas, more than 2000 times that step, so they never fight with it. Setting n = 0.001 makes Δd 100 times larger ($4.7 \cdot 10^{-4}$). When I set near = 0.001 with the slider I saw no visible change. This matches the estimate: even the coarser step is still 20 times smaller than the 0.01 gap between the curves and the canvas, and no two surfaces in this scene are closer than that. Moving the near plane in costs far more precision than moving the far plane out; in a deeper scene the same setting would cause z-fighting.

5 · Part D — Curves over the painting

Bézier basis. A cubic Bézier segment weights its four control points with the Bernstein polynomials:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t\,P_1 + 3(1-t)\,t^2 P_2 + t^3 P_3, \qquad t \in [0,1]$$

The four weights are non-negative and sum to $(t + (1 - t))^3 = 1$, so every point is an affine combination of the control points and the curve stays inside their convex hull.

It starts at P_0 and ends at P_3. At t = 0 every term with a factor t vanishes and $(1 - 0)^3 = 1$, so $B(0) = P_0$. At t = 1 every term with a factor (1 − t) vanishes, so $B(1) = P_3$.

Tangents and the C^1 condition. Differentiating gives

$$B'(t) = 3(1-t)^2(P_1 - P_0) + 6(1-t)\,t\,(P_2 - P_1) + 3t^2(P_3 - P_2)$$

so $B'(0) = 3(P_1 - P_0)$ and $B'(1) = 3(P_3 - P_2)$. Two segments P and Q share the joint $Q_0 = P_3$ (C^0). They are C^1 when the tangents also agree, $3(P_3 - P_2) = 3(Q_1 - Q_0)$, that is

$$Q_1 = 2P_3 - P_2$$

P_2, the joint and Q_1 are collinear, with the joint exactly in the middle. Because every segment covers an equal share 1/n of the global parameter u, dB/du = n · dB/dt on both sides, so C^1 in t is also C^1 in u. The editor enforces it while you drag. Moving a joint carries its two handles. Moving a handle mirrors the opposite handle through the joint (`c1NextControl`). The panel prints $|B'(1^-) - B'(0^+)|$ at each joint, which stays at 0 (up to 10^{-16}, see S-D1).

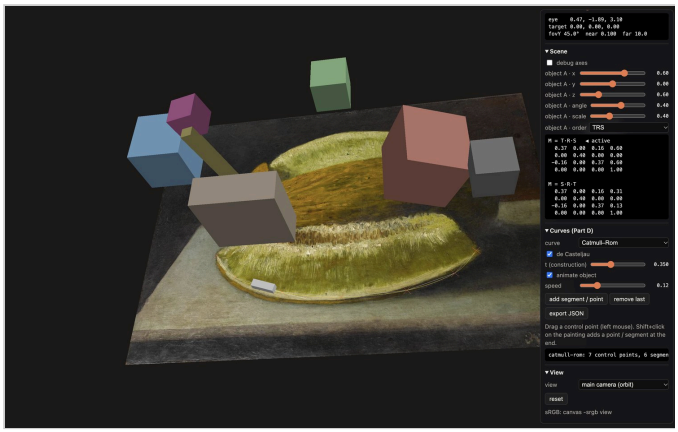
de Casteljau. For a parameter t the construction linearly interpolates neighbouring points level by level: 4 → 3 → 2 → 1. The last point is B(t). The app draws level 1 in cyan, level 2 in magenta and the point on the curve in green, for an adjustable t on the segment that contains it.

Second family: Catmull–Rom. Each segment between p_1 and p_2 uses tangents $(p_2 - p_0)/2$ and $(p_3 - p_1)/2$, so the curve passes through every control point and is C^1 by construction. The first and last points are duplicated so that the curve starts and ends on them.

Tracing the painting. The Bézier curve (3 segments, 10 control points) follows the upper silhouette of the whole melon. The Catmull–Rom curve (7 points) follows the lower edge of the cut slice. Control points are stored in normalised painting coordinates and mapped to the painting plane with x = (2u − 1)·a, y = 1 − 2v. Picking projects each point with P·V and takes the nearest one within 12 pixels. Dragging intersects the camera ray through the mouse with the plane z = 0.01. The map is affine, and Bézier and Catmull–Rom curves are invariant under affine maps, so evaluating in world space gives exactly the traced curve.



S-D1 · C^1 Bézier along the melon's top, de Casteljau at t = 0.35, joint readout.



S-D2 · Catmull–Rom along the lower edge of the slice, with the animated box.

Animation along the curve. A box moves along the active curve at constant speed. An arc-length table of 256 samples maps the distance travelled to the parameter u. The box is oriented by the tangent: rotateZ(atan2(dy, dx)) turns its long x axis onto B′. The white line shows the tangent.

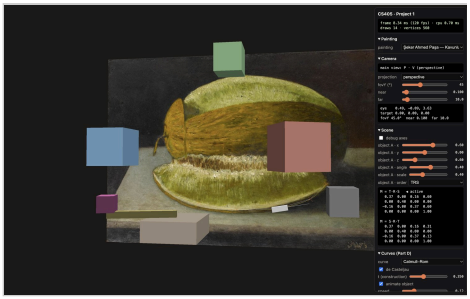
Export. "export JSON" writes the control points in normalised coordinates, u = (x + a)/(2a) and v = (1 − y)/2, origin at the top-left. Both files are in the repository under `exports/` and in the Appendix.

6 · Part E — Integration and performance

Parts A–D run in one page. The code is split into `math/` (vec3, mat4, curves; 51 unit tests with `npm test`), `scene/` (scene graph, objects, camera, curve editor, export), `render/` (WebGPU setup, painting, line batches, WGSL) and `ui/` (panel, stats, input). The README lists every control. The panel switches between the main and the inspection view, resets the scene and toggles the debug axes and the de Casteljau drawing. It also reads out the frame time, the CPU time per frame, the draw calls and the vertices submitted per frame. Every object has its own uniform buffer, and every line batch has its own vertex and uniform buffer: `queue.writeBuffer` runs before the frame's commands, so objects that shared one buffer would all be drawn with the last matrix written.

Measured on a MacBook Pro (Apple M3, integrated GPU), Chrome, canvas 2560 × 1468 device pixels, 120 Hz ProMotion display; values read from the panel (smoothed over many frames) in the screenshots above:

Scenario	Shot	Frame (ms)	CPU (ms)	Draw calls	Vertices / frame
Main view, perspective, Bézier + de Casteljau	S-C1	8.35	0.52	18	508
Main view, orthographic, Bézier + de Casteljau	S-C2	8.34	0.48	18	508
Main view, Catmull–Rom, debug axes on	S-B1	8.27	0.55	20	596
Main view, Catmull–Rom	S-E1	8.34	0.70	14	560



S-E1 · stats readout (top of the panel).

The frame time sits at the display's refresh interval (8.3 ms at 120 Hz) in every case, because `requestAnimationFrame` is synchronised to it: the GPU never limits this scene. The more informative number is the CPU time: building and submitting a frame takes about 0.5–0.7 ms, under 10 % of the 8.3 ms budget. The debug axes add 6 draw calls of 6 vertices; Catmull–Rom needs fewer draws because there is no de Casteljau construction to draw.

7 · What went wrong and how I found it

I had encountered a few bugs.

- **Non-uniform scale inside the chain.** The pedestal's local matrix first contained S(0.4, 0.15, 0.4). Every child inherited it, so the arm and the ornament came out flattened to 15 % of their height. It showed up in debug mode: the children's y axes were short. Fix: keep T·R in `local` and multiply the shape on the right only when drawing.
- **Camera jumped after reset.** Reset restored the camera but not the orbit angles, so the first drag recomputed the eye from the old angles. Fix: reset the orbit state too.
- **Orthographic view ignored the zoom.** The ortho box had a fixed height of 2.2, so the mouse wheel did nothing. Fix: derive its height from the orbit radius and fovY.
- **Degenerate lookAt.** With up parallel to the view direction the cross product is zero and the view matrix fills with NaN. Fix: the guard described in Part C.

AI usage

I did use AI for bugs that I couldn’t fix by myself and for the moments I felt stuck. It was a minimal usage.

Appendix · Exported curves (JSON)

`exports/curve-kavunlu-bezier-c1.json`

```
{"painting": "kavunlu", "kind": "bezier-c1", "segments": [
  [[0.3, 0.3], [0.315, 0.21], [0.39, 0.13], [0.48, 0.112]],
  [[0.48, 0.112], [0.57, 0.094], [0.68, 0.115], [0.76, 0.16]],
  [[0.76, 0.16], [0.84, 0.205], [0.885, 0.3], [0.885, 0.42]]]}
```

`exports/curve-kavunlu-catmull-rom.json`

```
{"painting": "kavunlu", "kind": "catmull-rom", "points": [
  [0.185, 0.625], [0.24, 0.715], [0.36, 0.78], [0.5, 0.81],
  [0.65, 0.8], [0.76, 0.75], [0.815, 0.675]]}
```